

Interruptible Agents Hackathon — Implementation Plan

2026-09-22 · @Someone · Deadline: 2026-09-25

1. Architecture recap

Three components, all running inside one asyncio event loop (the harness gives us `in_queue` / `out_queue`):

- **Fast path** — rule-based only, zero network calls. Acknowledges, asks clarifications, sends fillers. Must produce a spoken action within 800 ms of end-of-turn or an interruption (full credit), degrading to zero at 2500 ms.
- **Slow path** — Gemini calls (text, audio, vision) for intent/slot extraction, tool argument building, and phrasing. Every call has a timeout, with a deterministic rule-based fallback on timeout/error.
- **Coordination layer** — our own bookkeeping: current slots + intent, a `pending_calls` dict (`call_id` → tool, args, plan_version), a plan-version counter, and the tool manifest. Handles cancellation, re-planning, and duplicate-write prevention.

flowchart LR

```
IN[in_queue: events] --> FP[Fast path\nrule-based]
IN --> CO[Coordination layer\nslots, call_ids, plan version]
CO --> SP[Slow path\nGemini: text/audio/vision]
FP --> OUT[out_queue: actions]
SP --> OUT
CO --> OUT
```

Event types in: `tool_manifest`, `user_speech_chunk`, `user_audio_chunk`, `video_frame`, `interruption`, `tool_result`, `scenario_end`. Action types out: `filler_speech`, `tool_call`, `cancel_tool`, `clarification_request`, `final_response` (always carries `state_snapshot`).

2. Locked decisions

Area	Decision
Model	Gemini for all slow-path reasoning (text, audio, vision unified where possible). Every call wrapped in a ~1.5s timeout with a deterministic rule-based fallback (regex/keyword slot extraction + templated response) on timeout or error.
Audio	Send the raw MP3 directly to a multimodal Gemini call for transcription + intent in one shot — no separate local Whisper step. On failure, emit a <code>clarification_request</code> rather than guessing.
Image embedding	Gemini Embedding 2, called in parallel with the vision-reasoning call when a <code>video_frame</code> arrives. It's a bonus checkpoint only — omit the field if the call errors, never fake it.
Fast path	Pure rule-based/templated, zero LLM calls, zero network I/O. Fillers are template strings filled with whatever slot value is already known synchronously (e.g. "Looking into {city}...").
Speculation	No speculative tool calls on partial/mid-turn text. Wait for <code>end_of_turn</code> (text) or explicit confirmation (audio ambiguity) before firing any tool call. The slow-path LLM parse may start eagerly on partial text so slot-filling is warm by <code>end_of_turn</code> , but no <code>tool_call</code> fires before then.
Idempotency	Never re-emit a state-modifying call for an intent that already has a pending or completed <code>call_id</code> with the same args.
Cancellation	Explicit <code>cancel_tool</code> action on any interruption, within the 800 ms grace period — never just silently discard a stale result.

Area	Decision
Re-plan	Cancel/invalidate stale in-flight work, diff old vs new slots, keep results still valid, re-run stale reads, handle stale writes by confirming or amending, continue under a new plan version with fresh call_ids.

3. Repo layout

Must match the kit's submission structure exactly (`agent/agent.py` is the entry point declared in `submission.yaml`). Proposed internal module split inside `agent/` :

```
agent/
  agent.py          # ParticipantAgent – the event loop only, wires
                    # everything below
  coordination.py    # slots, plan_version, pending_calls dict, call_id
                    # gen, dup-write guard
  fast_path.py       # rule-based intent sniff, templated
                    # fillers/acks/clarifications, no I/O
  slow_path.py       # Gemini client wrapper: text intent+slots, audio,
                    # vision, embedding – all timeout-guarded with fallback
  tool_router.py     # schema-driven arg builder from tool_manifest,
                    # generic to unseen tools
  fallback.py        # deterministic regex/keyword extraction used when
                    # Gemini times out or errors
  submission.yaml
  harness/ scenarios/ docs/ run_local.py eval_submission.py # unchanged from
                    # the kit
```

One shared `AgentState` object (slots, intent, plan_version, pending_calls, manifest) is passed into `fast_path`, `slow_path`, and `tool_router` — no hidden global state, so any of the three of us can unit-test our module against a mocked state.

4. Work split (3 people)

Owner	Modules	Covers
Person A — Coordination & event loop	<code>agent.py</code> , <code>coordination.py</code>	The main loop, <code>AgentState</code> , plan-version tagging, <code>pending_calls</code> , dup-write guard, <code>scenario_end</code> tail handling. Everyone else's code plugs into this, so it should be the first thing scaffolded (stub interfaces day 1) even before it's fully correct.
Person B — Fast path & tool routing	<code>fast_path.py</code> , <code>tool_router.py</code> , <code>fallback.py</code>	Rule-based ack/filler/clarification templates; schema-driven argument builder from <code>tool_manifest</code> (generic to unseen tools, tested against <code>pub_09</code> + the <code>unseen_tool</code> generator template); the deterministic fallback used when Gemini fails.
Person C — Slow path & multimodal	<code>slow_path.py</code>	Gemini client wrapper for text intent/slot extraction, native audio calls, vision calls for frames, Gemini Embedding 2 calls — all async, all timeout- guarded, all with a fallback path into <code>fallback.py</code> .

Build order (per WALKTHROUGH.md), same order for all three modules in parallel:

1. `pub_01` / `pub_04` — turn → tool → grounded answer, or no tool at all
2. `pub_02` — interruption: filler, `cancel_tool` , new state, re-plan
3. `pub_03` / `pub_08` — chained calls (search → book), retry a read-only failure
4. `pub_05` / `pub_06` — audio: ambiguity + self-repair
5. `pub_07` — visual: frame-grounded manual lookup + embedding bonus
6. `pub_09` — unseen tool via manifest only

Daily sync recommended given the Sep 25 deadline — Person A's interfaces are the integration point, so changes there should be flagged in the group chat immediately.

5. Implementation checklist

- ☐ **A** — `AgentState` dataclass: slots, intent, plan_version, pending_calls, manifest
- ☐ **A** — Event loop skeleton reading `in_queue`, dispatching by `event_type`
- ☐ **A** — `call_id` generator + pending-calls tracking (tool, args, plan_version, status)
- ☐ **A** — Interruption handling: cancel stale calls, bump plan_version, diff slots
- ☐ **A** — `scenario_end` tail handling (finish pending work within 6s, never leave a call abandoned)
- ☐ **A** — Duplicate-write guard: check pending_calls before emitting any state-modifying tool_call
- ☐ **B** — Fast-path templates: acknowledgment fillers (content-aware, slot-filled), clarification requests
- ☐ **B** — Schema-driven arg builder reading `tool_manifest` (handles required/enum/nested object/array)
- ☐ **B** — Fallback slot extractor (regex/keyword) for when Gemini times out
- ☐ **B** — pub_09 unseen-tool test + `scenario_gen --template unseen_tool` test
- ☐ **C** — Gemini text call: intent + slot extraction from buffered `end_of_turn` text
- ☐ **C** — Gemini native audio call: transcription + intent in one call, with clarification fallback on low confidence
- ☐ **C** — Gemini vision call for `video_frame`: identify what's being referred to (e.g. port type)
- ☐ **C** — Gemini Embedding 2 call for `image_embedding`, run in parallel, non-blocking
- ☐ **C** — Timeout wrapper (~1.5s) around every Gemini call, routing to `fallback.py` on failure
- ☐ **All** — pub_01, pub_04 passing locally
- ☐ **All** — pub_02 passing (cancel + re-plan + updated snapshot)
- ☐ **All** — pub_03 passing (chained search → book, single booking only)
- ☐ **All** — pub_08 passing (retry read-only failure once)
- ☐ **All** — pub_05, pub_06 passing (ask-before-acting, repaired value wins)
- ☐ **All** — pub_07 passing (correct port + embedding bonus + no other ports mentioned)
- ☐ **All** — pub_09 passing (weather_lookup or equivalent via manifest only)

6. Testing & submission checklist

- ☐ Register Gemini API key on the event portal as a `SECRET_*` env var (do this early — key isn't registered yet as of writing)
- ☐ Run every scenario at `--time-scale 1` before trusting a number (scale 8 hides real API latency)
- ☐ Generate and test against unseen scenarios: `python -m harness.scenario_gen --template search_interrupt --n 10` and `--template unseen_tool --n 5`
- ☐ Confirm no synchronous/blocking calls in `run()` — all I/O via async client or `asyncio.to_thread`
- ☐ Confirm every `tool_call` sets its own `call_id`
- ☐ Confirm every `final_response` carries a `state_snapshot`
- ☐ Confirm no fillers repeat verbatim, and stay under ~4 per scenario
- ☐ Confirm nothing reads `ground_truth`, scenario IDs, or timestamps to decide behavior
- ☐ Fill in `submission.yaml`: team name, entry_point, python version, requirements (public PyPI only), env var names
- ☐ `python eval_submission.py . --time-scale 8 --reps 1` — fast import/boot/score check
- ☐ `python eval_submission.py . --reps 3` — full official procedure, real time
- ☐ Double-check no secrets committed to the repo
- ☐ Submit before Sep 25 deadline — one final package per team, no resubmission

7. Open risks

- Gemini API reliability is a known past pain point — the timeout+fallback design mitigates this, but budget test time specifically for the fallback path, not just the happy path.
- Eval machine's GPU/memory is undocumented — this is part of why audio/vision route through the Gemini API rather than a local model, avoiding the unknown.
- "One final submission per team" vs. an import-error costing "one of your daily submissions" contradict each other in the kit docs — worth asking organizers directly rather than assuming.
- Total per-scenario time budget is tight: tool calls (0.6-3s each) plus multiple Gemini calls (0.3-2s each) have to fit inside the 6s tail after the last user event, or pending work gets abandoned and fails checkpoints.

See the **Reference** tab for tool schemas, slot names, error codes, and the scoring rubric.